

CNN Documentation — 1D Temporal CNN on Raw Step Windows

WalkSensePlace

The tree models (XGBoost/RF) use 48 hand-engineered per-step features. The **1D-CNN does not** — it is fed the raw multichannel waveform of each step and learns its own shape/texture detectors. So there is no feature list; instead this covers (1) the input channels, (2) window preparation, (3) the architecture and what it learns, and (4) training/evaluation. It applies to both notebooks — only the number of output classes differs (3 for grass/loose/paved, 2 for soft/hard).

1 Inputs — 9 channels $\times$ 64 time-steps

Each step is a 64×9 array: $C = 9$ signal channels sampled at $T = 64$ time-points across the gait cycle.

#	channel	what it is	how derived
1	heel_sum	total heel pressure	sum of the 2 heel pads (pressure_08 , pressure_11)
2	fore_sum	total forefoot pressure	sum of the 5 forefoot pads (pressure_02/04/05/06/09)
3	total_p	total plantar pressure	sum of all 12 pressure pads
4–6	accel_x/y/z	tri-axial acceleration	raw accelerometer, per axis
7–9	gyro_x/y/z	tri-axial angular rate	raw gyroscope, per axis

The magnetometer is dead (all zeros) and unused. The CNN receives the *three pressure aggregates* (heel / forefoot / total) rather than all 12 pads individually — this keeps the input compact and focuses it on the front-to-back loading pattern, while the accel/gyro axes provide the raw vibration and rotation to find texture in.

2 Window preparation

- **Step segmentation.** The same **find_step_windows** used by the tree pipeline detects gait cycles (heel-strike to heel-strike); each window is one step. Windows shorter than MIN_DP samples are dropped; the label is the majority surface tag in the window (single-surface steps only).
- **Resample to fixed length.** Steps vary in duration, so each channel is linearly resampled to exactly $T = 64$ points, time-normalising the gait cycle so a fast and a slow step are comparable.
- **Per-channel z-scoring.** Each channel c of each step is standardised individually,

$$\tilde{x}_c[t] = \frac{x_c[t] - \mu_c}{\sigma_c}, \quad \mu_c = \frac{1}{T} \sum_t x_c[t], \quad \sigma_c = \sqrt{\frac{1}{T} \sum_t (x_c[t] - \mu_c)^2}. \quad (1)$$

This removes magnitude and keeps *shape* only — the same session/person invariance as the tree model’s normalised features. The CNN sees *how* pressure and motion evolve through the step, not how large they are, so it doesn’t key on body weight or insole tightness.

3 Architecture — what the network learns

The input $\tilde{x} \in \mathbb{R}^{T \times C}$ is transposed to C channels $\times T$ time and passed through a convolutional body then a classifier head.

1D convolution. A `Conv1d` layer with kernel size K maps C_{in} input channels to C_{out} output channels; each output channel o is a learned temporal filter,

$$y_o[t] = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{k=0}^{K-1} W_{o,c,k} x_c[t+k-\lfloor K/2 \rfloor], \quad (2)$$

followed by batch normalisation, a $\text{ReLU}(z) = \max(0, z)$ nonlinearity, and (in the first two blocks) max-pooling $\text{MaxPool}_2(y)[t] = \max(y[2t], y[2t+1])$, which halves the time axis and adds small shift-tolerance.

Body (feature extractor).

- `Conv1d(9→32, K=7) → BatchNorm → ReLU → MaxPool2` (64 → 32): 32 low-level temporal patterns across all 9 channels (e.g. an impact spike, a loading ramp).
- `Conv1d(32→64, K=5) → BatchNorm → ReLU → MaxPool2` (32 → 16): 64 mid-level patterns.
- `Conv1d(64→64, K=3) → BatchNorm → ReLU`: 64 higher-level patterns.
- **Global average pooling** collapses each filter’s response over time to one number,

$$z_o = \frac{1}{T'} \sum_{t=1}^{T'} h_o[t], \quad o = 1, \dots, 64, \quad (3)$$

giving a 64-dimensional learned summary z of the step.

Head (classifier).

$$\hat{y} = \text{softmax}\left(W_2 \text{ReLU}(W_1 \text{drop}_{0.3}(z) + b_1) + b_2\right), \quad \text{softmax}(u)_c = \frac{e^{u_c}}{\sum_{c'} e^{u_{c'}}}, \quad (4)$$

with a second dropout of 0.3 before the final linear layer. So the CNN’s “features” are the 64 learned convolutional filters — the data-driven analog of the hand-crafted features. Conceptually they can capture the same physical things (impact sharpness like `accel_jerk_ratio`, loading-curve shape like `gu_dip_depth`, rotational texture like `gyro_x_std`), just learned as temporal filters rather than computed formulas.

4 Training & evaluation

- **Loss.** Weighted cross-entropy over N steps,

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^N w_{y_n} \log \hat{y}_{n, y_n}, \quad w_c = \frac{\bar{w}_c}{\frac{1}{C} \sum_{c'} \bar{w}_{c'}}, \quad \bar{w}_c = \left(\frac{N}{C n_c}\right)^\gamma, \quad (5)$$

where n_c is the count of class c and $\gamma = \text{CLASS_WEIGHT_STRENGTH}$ is the same minority up-weighting dial used by the trees ($\gamma=0$ no weighting, $\gamma=1$ fully balanced).

- **Optimiser.** Adam, learning rate 10^{-3} , weight decay 10^{-4} .
- **Early stopping.** Up to 80 epochs; keep the weights with the best macro- F_1 on a *grouped* internal validation split (whole held-out recordings), patience 12. `drop_last` avoids a size-1 final batch that would break BatchNorm.
- **Cross-validation.** `StratifiedGroupKFold` grouped by recording/user, identical to the tree models, so out-of-fold scores are directly comparable. Always compare against the *grouped* (not shuffled) XGBoost focus-class F_1 .

5 Two honest limitations

1. **It cannot see cross-step features.** Each window is an independent, z-scored step, so the CNN structurally has no access to step-to-step variability — yet the tree model’s single strongest feature is exactly that (`orient_rest_a_roll3_std`, the rolling std of foot tilt across consecutive steps). A CNN over single steps cannot reconstruct a quantity defined across steps, so it is missing the most discriminating signal the trees rely on.
2. **Data size.** With a few thousand steps and the minority class in only a handful of recordings, a CNN overfits easily. If its grouped focus-class F_1 doesn’t beat XGBoost, that is the honest result, not a tuning failure — and given (1), a tie or slight loss is the expected outcome.

One-line reading

The CNN learns temporal filters over the raw heel/forefoot/total-pressure and accel/gyro waveforms of each step (time-normalised and z-scored, so it reads *shape*) and classifies from a global-average summary of those filters. It captures the same compliance/texture information the engineered features do, but — seeing each step in isolation — it misses the cross-step variability that is the merged tree model’s strongest cue, which is why the trees remain the model to beat.